

1. Mudança estrutural (Arquitetura do código)

Código antigo

O código antigo é um **script monolítico**, com praticamente toda a lógica implementada dentro de loops principais.

Problemas observados:

- Muitas responsabilidades concentradas em um único bloco
- Baixa reutilização de código
- Dificuldade para testar partes isoladas
- Manutenção complexa

Fluxo simplificado:

```
for empresa:
    for pdf:
        extrair texto
        limpar texto
        analisar texto
        gerar relatório
        salvar excel
```

Todas essas operações estavam misturadas no mesmo bloco.

Código novo

O código novo introduz **modularização através de funções**.

Funções criadas:

```
extrair_texto_pdf()
limpar_texto()
dividir_texto()
analisar_texto()
```

Isso estabelece separação clara de responsabilidades:

Função	Responsabilidade
extrair_texto_pdf	leitura do PDF
limpar_texto	limpeza textual
dividir_texto	otimização de tamanho do texto
analisar_texto	análise linguística

Essa mudança melhora significativamente a arquitetura do sistema.

2. Redução da complexidade de filtragem

Código antigo

A filtragem de regras era feita através de uma grande sequência de comparações:

```
erros = [match for match in matches if match.ruleId == 'O_FACTO_DA_ACÇÃO'
or match.ruleId == 'LINKING_VERB_PREDICATE_AGREEMENT'
or match.ruleId == 'GENERAL_VERB_AGREEMENT_ERRORS'
...
]
```

Essa lista possui dezenas de condições.

Problemas:

- baixa legibilidade
 - difícil manutenção
 - menor eficiência
-

Código novo

Foi introduzido um **conjunto de regras válidas**:

```
REGRAS_VALIDAS = {
'O_FACTO_DA_ACÇÃO',
'LINKING_VERB_PREDICATE_AGREEMENT',
...
}
```

Depois filtrado assim:

```
erros = [m for m in matches if m.rule_id in REGRAS_VALIDAS]
```

Vantagens:

- código mais curto
 - maior clareza
 - melhor desempenho
 - manutenção mais simples
-

3. Melhoria no processamento de texto

Código antigo

A limpeza de texto era feita manualmente:

```
bad = ['\n', '\r', '\t', '1', '2', '3', '4', '5', ...]
for name in bad:
    text = text.replace(name, '')
```

Problemas:

- muitas operações sequenciais
 - código longo
 - manutenção difícil
-

Código novo

Foi adotado **regex** para limpeza:

```
texto = re.sub(r'[\n\r\t]', ' ', texto)
texto = re.sub(r'[0-9*/%$+~-]', '', texto)
```

Benefícios:

- maior eficiência
 - código mais compacto
 - manutenção mais simples
-

4. Otimização importante: divisão do texto

Uma das melhorias mais relevantes.

Código antigo

O texto inteiro era enviado para análise:

```
matches = tool.check(text)
```

Problemas:

- textos grandes podem travar o servidor
 - risco de timeout
 - maior consumo de memória
-

Código novo

O texto é dividido em blocos:

```
def dividir_texto(texto, tamanho=20000):
    return [texto[i:i+tamanho] for i in range(0, len(texto), tamanho)]
```

Cada bloco é analisado separadamente.

Benefícios:

- maior estabilidade
 - redução de falhas
 - suporte melhor para PDFs grandes
-

5. Tratamento de erros

Código antigo

Tratamento limitado:

```
except lt.utils.LanguageToolError
```

Código novo

Foi implementada tentativa de reinicialização do servidor:

```
except lt.LanguageTool:  
    tool = lt.LanguageTool("pt-BR")
```

Isso aumenta a robustez da aplicação.

No entanto, existe um problema técnico nessa exceção (discutido mais adiante).

6. Uso correto de caminhos do sistema

Código antigo

Concatenação manual de strings:

```
path1 = path + "/" + caminho
```

Problemas:

- incompatibilidade potencial com Windows
 - menor segurança
-

Código novo

Uso de `os.path.join()`:

```
os.path.join(path, empresa)
```

Essa é a prática recomendada para manipulação de caminhos.

7. Escrita de arquivos

Código antigo

Arquivos abertos manualmente:

```
texto_txt = open(...)
texto_txt.write(text)
texto_txt.close()
```

Riscos:

- arquivos podem permanecer abertos em caso de erro
 - maior chance de vazamento de recursos
-

Código novo

Uso de gerenciador de contexto:

```
with open(path_texto_original, 'w', encoding='utf-8') as f:
    f.write(texto)
```

Isso garante fechamento automático do arquivo.

8. Organização geral do código

O novo código possui separação clara de seções:

```
CONFIGURAÇÃO
FUNÇÕES
PLANILHAS
ESCOLHA DOS CAMINHOS
LOOP PRINCIPAL
SALVAR RESULTADOS
```

Isso melhora significativamente:

- leitura
 - manutenção
 - organização
-

9. Redução de tamanho do código

Estimativa aproximada:

Código	Linhas
Código antigo	aproximadamente 700
Código novo	aproximadamente 350

Redução aproximada de 50%.

Código menor tende a resultar em:

- menor probabilidade de bugs
- manutenção mais simples

10. Problema introduzido no novo código

Existe um erro potencial aqui:

```
except lt.LanguageTool:
```

Essa não é uma exceção válida da biblioteca.

O correto provavelmente seria:

```
except Exception:
```

ou

```
except lt.utils.LanguageToolError:
```

Isso deve ser corrigido para evitar falhas inesperadas.

11. Comparação geral

Critério	Código antigo	Código novo
Arquitetura	Monolítica	Modular
Legibilidade	Baixa	Alta
Performance	Média	Melhor
Manutenção	Difícil	Mais simples
Tratamento de erro	Limitado	Melhor
Processamento de texto	Manual	Regex
Robustez	Média	Maior

Critério	Código antigo	Código novo
Escalabilidade	Baixa	Moderada

12. Avaliação técnica final

Avaliação aproximada de qualidade:

Versão	Avaliação
Código antigo	5/10
Código novo	8.5/10

13. Melhorias adicionais possíveis

Para elevar o código a um nível mais avançado, seria possível implementar:

1. Paralelização do processamento

Uso de:

`multiprocessing`

Permitiria analisar vários PDFs simultaneamente.

2. Barra de progresso

Uso da biblioteca:

`tqdm`

Melhora acompanhamento da execução.

3. Sistema de logs

Substituir `print()` por:

`logging`

Permite registrar erros e histórico de execução.

4. Arquivo de configuração

Separar parâmetros em arquivo externo, por exemplo:

`config.yaml`

Isso facilita ajustes sem alterar o código.

5. Estrutura de pipeline de dados

Arquitetura mais robusta poderia ser:

```
PDF
↓
Extração de texto
↓
Limpeza
↓
Análise linguística
↓
Banco de dados
↓
Relatórios
```

14. Avaliação geral

O novo código demonstra evolução clara em relação ao anterior:

- modularização
- melhor uso de bibliotecas
- maior eficiência
- organização estrutural superior

Essas mudanças indicam amadurecimento na prática de programação.

Se desejar, também posso mostrar **três melhorias estruturais que podem tornar esse script aproximadamente 20 vezes mais rápido**, especialmente quando analisando milhares de PDFs. Essas melhorias envolvem paralelização, uso otimizado do LanguageTool e melhorias no pipeline de processamento.